

INVOICEFLOW

Complete Master Roadmap

Zero → Production → Super-Hero SaaS

Implementation master document for building InvoiceFlow with Gemini or another coding agent. Includes product scope, architecture, setup, database, API, UI, templates, PDF, PWA, testing, deployment, monetization, launch and long-term roadmap.

Table of Contents

- 1. Product definition and vision
- 2. Problem, audience and positioning
- 3. Scope and MVP boundaries
- 4. Technology stack
- 5. Product architecture
- 6. Repository structure and setup
- 7. Database and domain model
- 8. Authentication and security
- 9. Frontend implementation
- 10. Backend implementation
- 11. Invoice calculation engine
- 12. Template system and customization
- 13. PDF generation
- 14. PWA/offline
- 15. API contract
- 16. UI/UX system
- 17. 14-day execution plan
- 18. Testing and QA
- 19. Deployment and operations
- 20. Monetization
- 21. Launch strategy
- 22. Future roadmap
- 23. Super-hero vision
- 24. Gemini execution protocol
- 25. Definition of done and master checklists

1. Product Definition and Vision

InvoiceFlow is a professional invoice-generation and management SaaS/PWA for freelancers, developers, consultants, creators, agencies and small businesses. It should feel like a focused combination of a beautiful document designer and practical invoicing software.

Core promise

A first-time user should be able to go from zero to a professional invoice in minutes: Create → Choose → Customize → Preview → Generate PDF → Download/Share.

MVP success condition

By Day 14, a real user can register, create a business profile, add a client, create an invoice, choose a free or premium template, customize it, preview it, generate a professional A4 PDF, download it and install InvoiceFlow as a PWA.

Non-negotiable architecture rule

Invoice data and invoice design must be separate. One invoice object must be renderable by multiple templates without changing the stored invoice data.

2. Problem, Audience and Positioning

Origin story

The product is based on a real freelance pain point: receiving a request for an invoice, not knowing the best way to produce a professional one, searching for tools, and deciding to build a focused solution instead.

Problems to solve

- Manual formatting is slow and inconsistent.
- New freelancers do not know what a professional invoice should contain.
- Generic document editors require design work.
- Many accounting suites are too complex for someone who simply needs a polished invoice.
- Freelancers need branding, hourly billing, tax/discount support, reusable clients and invoice history.

Target users

User	Main job	InvoiceFlow value
Freelancer	Bill clients	Fast branded invoice + PDF
Developer	Bill project/hourly work	Hourly/quantity billing + reusable clients
Consultant	Bill services	Professional layout + terms/payment details
Designer/creator	Send beautiful invoices	Premium templates + customization
Small agency	Repeat billing	History, duplicate, clients, branding
Small business	Basic invoicing	Business profile + client workflow

Positioning

Start as the beautiful, easy invoice builder; evolve into a lightweight financial workspace for freelancers and small businesses.

3. Scope and MVP Boundaries

MVP — ship in 14 days

- Authentication and protected routes
- Business profile and logo
- Client CRUD and client selection
- Invoice create/edit/delete/duplicate
- Invoice numbering, dates and due dates
- Quantity, hourly and fixed-price items
- Subtotal, discount, tax, extra charges and grand total
- Notes, terms, payment instructions, bank details, UPI and signature fields
- Five free templates
- Two premium templates
- Template switching and live preview
- Brand colors, logo, font and section visibility customization
- A4 PDF generation and download
- Invoice history, search, filter and statuses
- Installable PWA and offline draft support
- Production deployment and public launch

Do NOT let these features break the 14-day launch

- AI generation
- Recurring invoices
- Client portal
- Team accounts

- Accounting integrations
- WhatsApp automation
- Advanced analytics
- Multi-language
- Complex international tax engine
- Template marketplace
- Native apps
- Full subscription system
- OCR
- Expense management
- Bank reconciliation

Scope discipline

When a feature threatens the deadline, put it into the parking lot. A polished launched MVP is more valuable than a half-built super-app.

4. Technology Stack

Layer	Technology	Purpose
Frontend	Vue 3 + TypeScript	UI/application
Build	Vite	Development/build
Styling	Tailwind CSS	Responsive design system
Routing	Vue Router	SPA navigation
State	Pinia	Shared state
HTTP	Axios	API client
Icons	Lucide	Icons
PWA	Vite PWA tooling	Manifest/service worker
Backend	Laravel 13	API/business logic
Runtime	PHP 8.3+	Laravel runtime
Auth	Laravel Sanctum	SPA authentication
ORM	Eloquent	Database access
DB	MySQL	Persistence
Testing	Vitest	Unit/component tests
E2E	Playwright	Critical user flows
PDF	Open-source Laravel-compatible PDF library	A4 export
VCS	Git + GitHub	Version control

Use current official documentation at implementation time. Do not blindly pin versions from old tutorials. Keep the stack deliberately small for the 14-day sprint.

5. Product Architecture

Browser/PWA → Vue UI → Router/Pinia/Services → Laravel API → Form Requests/Policies → Services → Eloquent → MySQL

Invoice rendering architecture

Invoice record + Business + Client + Items + Customization
↓

Template Definition
↓
Template Renderer
↙ ↘
Live Preview PDF

Architecture rules

- Views orchestrate pages; reusable components handle UI.
- API calls live in services.
- Shared state lives in Pinia only when necessary.
- Business logic lives in Laravel services and domain utilities, not controllers.
- Validation belongs in Form Requests.
- Authorization belongs in Policies.
- Financial values must have deterministic rounding/decimal behavior.
- Never use frontend-only authorization.

6. Repository Structure and Zero-to-Setup

```
invoiceflow/  
├── frontend/  
│   ├── src/assets  
│   ├── src/components  
│   ├── src/composables  
│   ├── src/layouts  
│   ├── src/router  
│   ├── src/services  
│   ├── src/stores  
│   ├── src/templates  
│   ├── src/types  
│   ├── src/utils  
│   └── src/views  
├── backend/  
│   ├── app/Http/Controllers  
│   ├── app/Http/Requests  
│   ├── app/Http/Resources  
│   ├── app/Models  
│   ├── app/Policies  
│   ├── app/Services  
│   ├── database/migrations  
│   ├── database/seeder  
│   └── routes/api.php  
├── docs/  
└── README.md
```

Prerequisites

- Git/GitHub
- Node.js supported LTS/current compatible release
- npm
- PHP 8.3+
- Composer
- MySQL
- VS Code or equivalent
- Optional DB GUI

Bootstrap order

- Create GitHub repo and clone it.
- Create Vue + TypeScript + Vite frontend.
- Install Tailwind, Router, Pinia, Axios, Lucide and PWA tooling.

- Create Laravel backend.
- Create MySQL database.
- Configure Sanctum and SPA CORS/session settings.
- Run first migration.
- Add /api/health endpoint.
- Connect frontend to API.
- Commit clean baseline.

Environment

```
Frontend:
VITE_API_BASE_URL=
VITE_APP_NAME=InvoiceFlow

Backend:
APP_NAME=InvoiceFlow
APP_ENV=local
APP_KEY=
APP_URL=
DB_CONNECTION=mysql
DB_HOST=
DB_PORT=3306
DB_DATABASE=
DB_USERNAME=
DB_PASSWORD=
FRONTEND_URL=
SANCTUM_STATEFUL_DOMAINS=
```

7. Database and Domain Model

Table	Core fields
users	id, name, email, password, timestamps
business_profiles	user_id, name, email, phone, address, website, logo, GST/tax, bank, UPI
clients	user_id, name, company, email, phone, address, GST/tax, notes
invoices	user_id, client_id, number, status, dates, currency, subtotal, discount, tax, charges, total, notes, terms, payment, bank, UPI, signature, template_key, customization
invoice_items	invoice_id, description, quantity, unit_type, rate, amount, sort_order
invoice_templates	key, name, tier, price, metadata, active
payments	invoice_id, provider, order/payment IDs, amount, currency, status, paid_at (later/MVP-ready)

Relationships

```
User 1—1 BusinessProfile
User 1—many Clients
User 1—many Invoices
Client 1—many Invoices
Invoice 1—many InvoiceItems
Invoice 1—many Payments
```

Data design

Keep core money fields explicit and queryable. JSON is appropriate for flexible visual customization metadata, but not as a substitute for core financial columns.

8. Authentication and Security

Auth flow

```
Register/Login → Sanctum session → GET current user → protected Vue routes → authorized API requests
```

Security checklist

- Validate all input with Form Requests.
- Authorize every resource mutation with Policies.
- Prevent mass assignment.
- Use CSRF protection with SPA cookie authentication.
- Restrict CORS in production.
- Rate-limit sensitive endpoints.
- Validate uploaded logo/signature MIME, extension and size.
- Never expose secrets through VITE_ variables.
- Never trust client-side payment success.
- Use HTTPS in production.
- Avoid leaking secrets in logs/errors.
- Use ownership checks for user-scoped records.

9. Frontend Implementation

Routes

```
/
/login
/register
/dashboard
/invoices
/invoices/create
/invoices/:id
/invoices/:id/edit
/clients
/templates
/business
/settings
```

Core components

- App shell/sidebar/mobile navigation
- Form controls
- Client selector
- Invoice item editor
- Totals summary
- Template card
- Template preview
- Customization panel
- Invoice preview canvas
- Status badge
- Confirm modal
- Toast
- Loading/skeleton
- Empty state
- Error state

Core TypeScript types

```
User
BusinessProfile
Client
Invoice
InvoiceItem
InvoiceTotals
```

InvoiceCustomization
InvoiceTemplate
InvoiceStatus
Currency
PaymentDetails

State ownership

- Auth/session → auth store
- Business profile → business store or composable
- Clients → client store/query layer
- Invoice editor → page-local state/composable until save
- UI preferences → local storage/store
- Offline drafts → IndexedDB layer

10. Backend Implementation

Request pipeline

Route → Controller → FormRequest → Policy → Service → Eloquent → API Resource → JSON

Services

- InvoiceService
- InvoiceCalculationService
- PdfInvoiceService
- TemplateService
- BusinessProfileService
- FileUploadService
- PaymentService (later)

Controller rule

Controllers should coordinate HTTP concerns, not become giant business-logic files.

11. Invoice Calculation Engine

Supported billing

- Quantity × rate
- Hours × hourly rate
- Fixed-price item (quantity 1)

Calculation contract

```
lineAmount = quantity × rate
subtotal = sum(lineAmount)
discountAmount = fixed OR subtotal × percentage / 100
taxableAmount = subtotal - discountAmount
taxAmount = taxableAmount × taxPercentage / 100
grandTotal = taxableAmount + taxAmount + additionalCharges
```

Define rounding rules before implementation. Prefer decimal-safe handling or integer minor units for persisted monetary values where practical; do not allow inconsistent floating-point calculations between browser and server.

Statuses

- Draft
- Sent

- Pending
- Paid
- Overdue
- Cancelled (future)

12. Template System and Customization

Free templates

Template	Direction
Classic	Traditional business
Minimal	Whitespace/clean
Modern	Contemporary SaaS
Professional	Corporate
Freelancer	Creative professional

Premium templates

Template	Direction
Executive	High-end corporate
Luxury	Premium brand/editorial

Template metadata

```
{ key, name, tier, price, layout, font, colors, headerStyle, footerStyle, tableStyle, borderStyle, logoPosition, visibleSections, sectionOrder }
```

Customization MVP

- Logo
- Primary/secondary colors
- Font choice
- Header style
- Footer style
- Table style
- Logo position
- Show/hide business/client/items/tax/discount/payment/bank/notes/terms/signature
- Section ordering where feasible

Templates must consume a normalized InvoiceViewModel so PDF and browser rendering can remain consistent.

13. PDF Generation

Requirements

- A4
- Correct margins
- Logo and brand colors
- Long text wrapping
- Multi-page invoices
- Stable table layout
- Page breaks
- Totals grouping

- Optional header/footer
- Print-friendly output

PDF test matrix

Scenario	Expected
1 item	One clean page
20+ items	Correct multi-page layout
Long description	Wraps without clipping
Large logo	Constrained
Long client/company name	No overflow
Tax + discount	Correct total
No logo	Balanced layout
2+ pages	Usable page breaks

14. PWA and Offline

MVP

- Manifest
- Icons
- Service worker
- Static shell caching
- Installability
- Offline fallback
- Local draft persistence using IndexedDB or suitable abstraction

Future sync

Edit offline → save local draft → connection returns → sync queue → conflict detection → server reconciliation

15. API Contract

Method	Endpoint	Purpose
POST	/api/auth/register	Register
POST	/api/auth/login	Login
POST	/api/auth/logout	Logout
GET	/api/user	Current user
GET/PUT	/api/business	Business profile
GET/POST	/api/clients	List/create clients
GET/PUT/DELETE	/api/clients/{id}	Client lifecycle
GET/POST	/api/invoices	List/create invoices
GET/GET/PUT/DELETE	/api/invoices/{id}	Invoice lifecycle
POST	/api/invoices/{id}/duplicate	Duplicate
GET	/api/templates	Template catalog
GET	/api/invoices/{id}/pdf	PDF
GET	/api/dashboard	Dashboard metrics

Use consistent success/error JSON shapes and field-level validation errors. Document expected status codes. Do not make the frontend guess response structure.

16. UI/UX System

Visual direction

- Premium SaaS feel
- Clean typography
- Compact but breathable spacing
- Accessible contrast
- Responsive mobile/tablet/desktop
- Subtle motion
- Strong empty/loading/error states

Editor layout

Desktop: Editor controls | Live A4 preview
Mobile: stacked/accordion editor sections + accessible preview

Reusable primitives

- Button
- Input
- Textarea
- Select
- Checkbox
- Switch
- Modal
- Drawer
- Card
- Badge
- Dropdown
- Toast
- Table
- Pagination
- Skeleton
- EmptyState

17. 14-Day Execution Plan

Day	Build	Exit criterion
1	Foundation: repo, Vue/TS/Vite, Laravel, MySQL, Sanctum, Tailwind, Router, Pinia, Axios, PWA baseline, env, health API.	Both apps run; DB/API connected.
2	Auth + application shell: landing, register, login, logout, protected routes, dashboard shell, mobile nav.	User can authenticate and enter app.
3	Business + clients: profile, logo foundation, client CRUD/search/select.	Reusable business/client data.
4	Invoice engine: model, items, numbering, dates, currency, tax, discount, totals.	Correct calculations.
5	Complete editor: notes, terms, payment, bank/UPI, signature, drafts, edit, delete, duplicate.	Complete invoice record.
6	Template engine + five free templates.	Same invoice renders in multiple designs.
7	Two premium templates + locked/premium UX.	Free/premium catalog works.

Day	Build	Exit criterion
8	Customization: logo, colors, fonts, layout/sections, live preview.	Premium editor experience.
9	PDF: A4, download, print, multi-page tests.	Professional PDF.
10	Dashboard: metrics, history, filters, search, status, edit, duplicate, download.	Usable daily workflow.
11	PWA: manifest, service worker, installability, offline shell, local drafts.	Installable PWA.
12	Premium architecture; Razorpay only if safe for deadline.	Monetization-ready.
13	QA: responsive, validation, accessibility basics, unit/E2E, security, polish.	Release candidate.
14	Deploy frontend/backend/DB, HTTPS, env, smoke tests, phone install, real invoice, launch.	Public launch.

Daily operating rule

At the end of each day: commit, run relevant checks, record known issues, and write the next day starting point. Do not carry untracked experimental work into the next milestone.

18. Testing and QA

Unit tests

- Line amount
- Subtotal
- Discount
- Tax
- Grand total
- Rounding
- Invoice numbering

Critical E2E

Register → Login → Business setup → Add client → Create invoice → Add items → Template → Customize → Preview → PDF → Download → Logout → Login → Verify invoice

QA matrix

- Phone
- Tablet
- Laptop
- Desktop
- Current Chrome
- At least one additional modern browser where practical
- Slow network
- No network
- Long invoice
- Empty data
- Validation errors

19. Deployment and Operations

Production sequence

- Provision MySQL
- Configure Laravel production env
- Configure secrets

- Run migrations
- Configure storage
- Configure CORS/Sanctum
- Build frontend
- Deploy frontend
- Deploy PHP/Laravel backend
- Enable HTTPS
- Configure API URL
- Smoke-test auth
- Smoke-test invoice
- Smoke-test PDF
- Install PWA on phone

Cost strategy

Target ₹0 mandatory development spend. Use free tiers/subdomains initially where technically suitable.

PHP/Laravel hosting may require a paid provider depending on current free-tier availability; verify current terms at deployment time. Buy a custom domain later if validation justifies it. Payments, email, storage and AI APIs should not be mandatory for the first launch.

20. Monetization

Free tier

- Five free templates
- Basic branding
- Client management
- Invoice history
- PDF generation

Premium tier

- Premium templates
- Advanced customization
- Future premium fonts/assets
- Future advanced export/features

Payment flow

Select premium → frontend requests order → Laravel creates order → payment gateway → gateway callback/response → Laravel server-side verification → purchase stored → template unlocked

Never unlock premium permanently based only on a frontend success callback. If payment integration risks Day 14, launch with locked premium UI and implement real payment immediately after launch.

21. Launch Strategy

Launch package

- Public URL
- GitHub repo
- README
- Architecture diagram
- Screenshots
- Short demo video
- PWA mobile screenshot

- Example invoice
- LinkedIn launch post
- Portfolio case study

Story

Tell the authentic story: a real freelance invoice need exposed a problem, so InvoiceFlow was built to make professional invoicing fast, simple and beautiful.

22. Future Roadmap

Phase	Features
V1.1	Password reset, email verification, more templates, onboarding, invoice defaults
V1.2	Recurring invoices, payment links, Razorpay/Stripe, reminders
V1.3	Client portal, public invoice links, online payments
V1.4	Revenue analytics, overdue automation, reports
V2	Teams, workspaces, roles and permissions
V2.1	Quotes, estimates, credit/debit notes, purchase orders
V2.2	Expenses, time tracking, projects
V3	Template marketplace/designer ecosystem
V3+	AI invoice creation, AI descriptions, AI reminders, AI insights
Long term	Accounting integrations, multi-country tax, multi-language, broader financial workspace

23. Super-Hero Vision

The long-term product should become more than an invoice generator: a lightweight operating system for freelancer/small-business finances.

Projects → Time Tracking → Clients → Quotes → Invoices → Payments → Expenses → Reports → Tax support → AI Assistant

AI possibilities

- Natural-language invoice creation
- Professional service-description rewriting
- Payment reminder generation
- Monthly revenue summaries
- Overdue-client insights
- Follow-up suggestions
- Natural-language invoice-history queries

Marketplace possibilities

Designers can submit professional invoice templates. After review, templates can be published and sold, creating a scalable design ecosystem and marketplace revenue.

24. Gemini Execution Protocol

Give this PDF to Gemini and tell it to treat it as the master specification. Gemini should act as a senior full-stack engineer and implementation partner.

Mandatory agent rules

- Read the full roadmap before coding.

- Follow the 14-day order unless dependencies force a change.
- For every feature identify DB + API + frontend + tests.
- Use official/current documentation for package behavior.
- Do not add unnecessary dependencies.
- Do not expose secrets.
- Do not delete working features to add new ones.
- Keep controllers thin and business logic in services.
- Use validation and authorization on every mutation.
- Run build/tests after meaningful milestones.
- Explain files changed and verification commands.
- Make reasonable assumptions instead of blocking on non-critical questions.
- Do not build V2 before MVP is launched.
- Maintain a checklist and milestone status.

Required Gemini response format

MILESTONE: Day X — Feature

1. Goal
2. Current state
3. Files to create/change
4. Implementation
5. DB/API changes
6. Tests
7. Verification commands
8. Known issues
9. Next step

When Gemini should stop and ask

- A destructive migration could cause data loss.
- A required product decision cannot be inferred safely.
- Credentials/API keys are required but unavailable.
- Two architectures have materially different long-term consequences.

25. Definition of Done and Master Checklists

MVP definition of done

- Register/login works
- Business profile works
- Client CRUD works
- Invoice CRUD works
- Calculations are correct
- Five free templates work
- Two premium templates work
- Customization works
- Live preview works
- PDF works
- Multi-page PDF works
- History/search/filter works
- PWA installs
- Offline draft works
- Critical security checks pass
- Critical E2E path passes

- Production URL works
- Real invoice can be created/downloaded from production
- README/setup docs exist
- Launch assets exist

Before coding

- Repo created
- Tools installed
- Vue app created
- Laravel app created
- MySQL created
- Env configured
- Health endpoint works
- Frontend reaches backend
- Baseline commit exists

Before launch

- Production DB
- Migrations
- HTTPS
- CORS/Sanctum
- Storage
- Auth
- Business
- Clients
- Invoices
- Calculations
- Templates
- Customization
- PDF
- PWA
- Mobile
- E2E
- Security
- README
- Smoke test

After launch

- Collect real feedback
- Track visitor→registration→first invoice→first PDF
- Fix bugs before major features
- Prioritize payments/recurring/client portal
- Then expand into teams, AI and marketplace

INVOICEFLOW NORTH STAR

Build → Test → Ship → Learn → Improve.

14-Day mission: launch a real, beautiful, installable invoice application solving a real business problem.

Engineering mission: maintain clean separation between UI, API, domain logic, persistence and templates.

Business mission: validate demand before adding complexity, then monetize premium design and workflow features.

Long-term mission: evolve InvoiceFlow into a complete financial workspace with payments, recurring billing, analytics, collaboration, marketplace and AI.